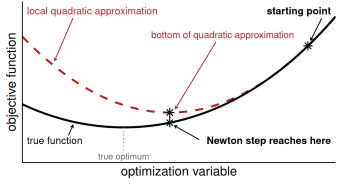
Optimization in Machine Learning

Second order methods
Newton-Raphson



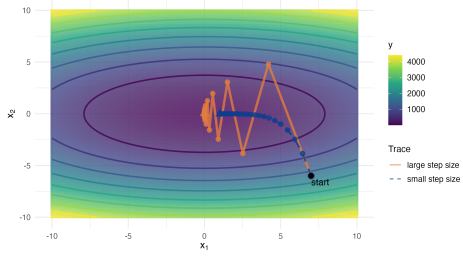
Learning goals

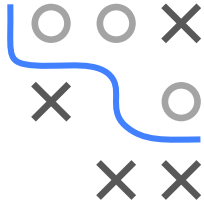
- Newton-Raphson
- Damped Newton-Raphson

Slides based on
▸ AGGARWAL2020 (Ch. 5.4-5.7)

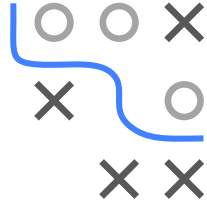
MOTIVATION: BEYOND GRADIENT INFORMATION

- **Recall:** GD only uses the gradient $\Rightarrow$ *ignores curvature*
- Hessian $\mathbf{H} = \nabla^2 f$ (curvature) encodes how fast the gradient itself changes along each direction
  - High curvature: gradient is unreliable further away $\Rightarrow$ take a *small* step
  - Low curvature: gradient stays valid further away $\Rightarrow$ take a *large* step
- GD only picks one global α for all directions $\Rightarrow$ for ill-conditioned $\mathbf{H}$, this causes zig-zagging / slow convergence:

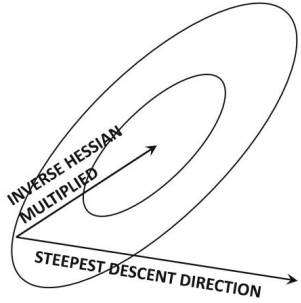




MOTIVATION: BEYOND GRADIENT INFORMATION



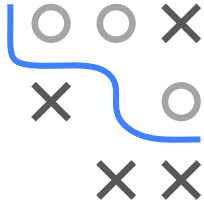
- **Idea:** Use full curvature information instead of a single scalar α
  - rescale the gradient *individually per direction* via $\mathbf{H}^{-1}$
  - biases updates towards low-curvature directions, where a large step is unlikely to overshoot or worsen f



▸ Click for source

Effect of multiplying the steepest-descent direction with the inverse Hessian

NEWTON-RAPHSON



- **Assumption:** $f \in \mathcal{C}^2$
- **Aim:** Find stationary point $\boldsymbol{x}^*$, i.e., $\nabla f(\boldsymbol{x}^*) = \mathbf{0}$
- **Idea:** Find root of first order Taylor approx of $\nabla f(\boldsymbol{x})$:

$$\nabla f(\boldsymbol{x}) \approx \nabla f(\boldsymbol{x}^{[t]}) + \nabla^2 f(\boldsymbol{x}^{[t]})(\boldsymbol{x} - \boldsymbol{x}^{[t]}) = \mathbf{0}$$

$$\nabla^2 f(\boldsymbol{x}^{[t]})(\boldsymbol{x} - \boldsymbol{x}^{[t]}) = -\nabla f(\boldsymbol{x}^{[t]})$$

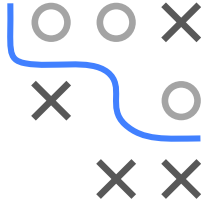
$$\boldsymbol{x}^{[t+1]} = \boldsymbol{x}^{[t]} - (\nabla^2 f(\boldsymbol{x}^{[t]}))^{-1} \nabla f(\boldsymbol{x}^{[t]})$$

- **Update scheme:**

$$\boldsymbol{x}^{[t+1]} = \boldsymbol{x}^{[t]} + \mathbf{d}^{[t]}$$

with $\mathbf{d}^{[t]} = -(\nabla^2 f(\boldsymbol{x}^{[t]}))^{-1} \nabla f(\boldsymbol{x}^{[t]})$

NEWTON-RAPHSON: PRACTICE

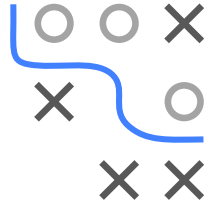


- **Note:** In practice, we get $\mathbf{d}^{[t]}$ by solving the linear system

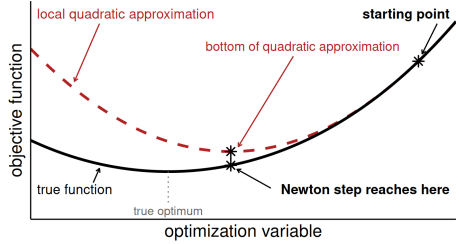
$$\nabla^2 f(\boldsymbol{x}^{[t]})\mathbf{d}^{[t]} = -\nabla f(\boldsymbol{x}^{[t]})$$

with direct (matrix decompositions) or iterative methods

NEWTON-RAPHSON: INTUITION



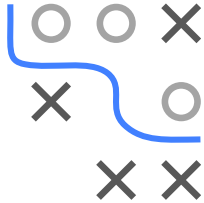
- Newton update approximates the function locally with a quadratic bowl and moves to the bottom of the bowl in a single step
- The “learning rate” is incorporated implicitly
- In general, the bottom of the quadratic bowl is not the bottom of the true function $\Rightarrow$ multiple Newton updates are usually necessary



Note: univariate functions are rather “boring” as there are only two possible directions $\Rightarrow$ we’ll move on to more interesting examples soon

- **Caveat**: if the quadratic approximation is poor (e.g., far from the current point), the Newton step may worsen f (see next subchapter)

ANALYTICAL EXAMPLE WITH QUADRATIC FORM



- Consider $f(x_1, x_2) = x_1^2 + \frac{x_2^2}{2}$
- Update direction: $\mathbf{d}^{[t]} = -(\nabla^2 f(x_1^{[t]}, x_2^{[t]}))^{-1} \nabla f(x_1^{[t]}, x_2^{[t]})$
- Gradient and Hessian:

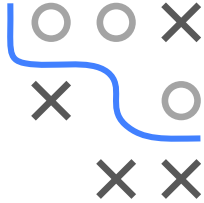
$$\nabla f(x_1, x_2) = \begin{pmatrix} 2x_1 \\ x_2 \end{pmatrix}, \quad \nabla^2 f(x_1, x_2) = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$$

- First step:

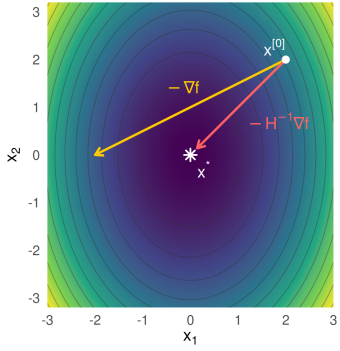
$$\begin{pmatrix} x_1^{[1]} \\ x_2^{[1]} \end{pmatrix} = \begin{pmatrix} x_1^{[0]} \\ x_2^{[0]} \end{pmatrix} + \mathbf{d}^{[0]} = \begin{pmatrix} x_1^{[0]} \\ x_2^{[0]} \end{pmatrix} - \begin{pmatrix} 1/2 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 2x_1^{[0]} \\ x_2^{[0]} \end{pmatrix}$$

$$= \begin{pmatrix} x_1^{[0]} \\ x_2^{[0]} \end{pmatrix} + \begin{pmatrix} -x_1^{[0]} \\ -x_2^{[0]} \end{pmatrix} = \mathbf{0}$$

ANALYTICAL EXAMPLE WITH QUADRATIC FORM

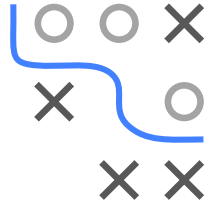


- Newton-Raphson only needs one iteration for quadratic forms
- See how the inverse Hessian rescales the gradient:
  - x_1: higher curvature $\Rightarrow$ scaled by $1/2$ by $\mathbf{H}^{-1}$
  - x_2: lower curvature $\Rightarrow$ scaled by 1 by $\mathbf{H}^{-1}$
- **Note**: For diagonal H, each dimension is scaled individually and we can easily interpret the scaling factors. For non-diagonal H, the update direction becomes a linear combination of all dimensions.

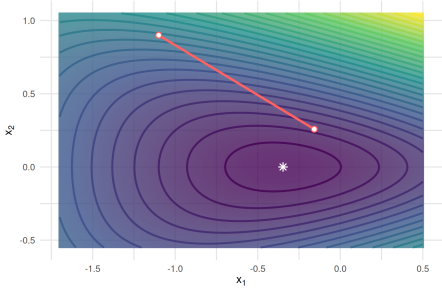


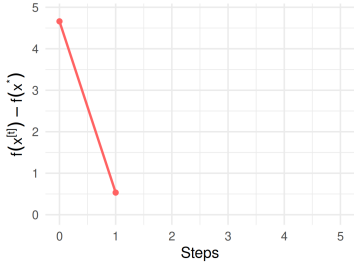
NEWTON-RAPHSON: NON-QUADRATIC EXAMPLE

$f(\boldsymbol{x}) = \exp(x_1 + 3x_2 - 0.1) + \exp(x_1 - 3x_2 - 0.1) + \exp(-x_1 - 0.1)$,
with $\boldsymbol{x}^* = (-\frac{1}{2}\log 2, 0)^\top$



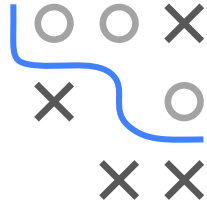
After 1 Newton step:



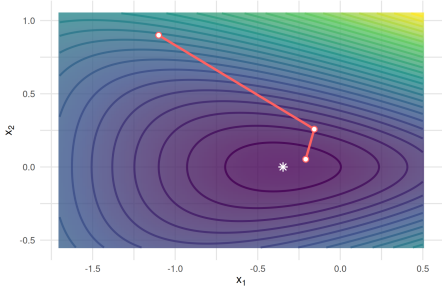


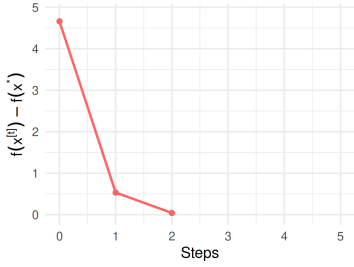
NEWTON-RAPHSON: NON-QUADRATIC EXAMPLE

$f(\boldsymbol{x}) = \exp(x_1 + 3x_2 - 0.1) + \exp(x_1 - 3x_2 - 0.1) + \exp(-x_1 - 0.1)$,
with $\boldsymbol{x}^* = (-\frac{1}{2}\log 2, 0)^\top$



After 2 Newton steps:

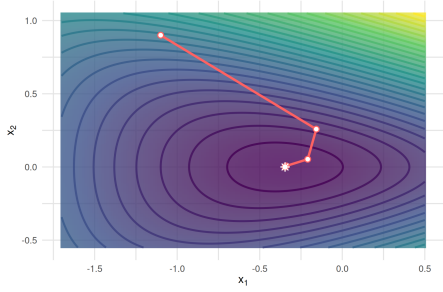


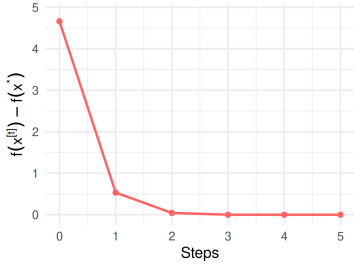


NEWTON-RAPHSON: NON-QUADRATIC EXAMPLE

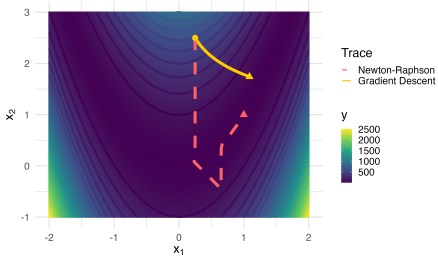
$f(\boldsymbol{x}) = \exp(x_1 + 3x_2 - 0.1) + \exp(x_1 - 3x_2 - 0.1) + \exp(-x_1 - 0.1)$,
with $\boldsymbol{x}^* = (-\frac{1}{2}\log 2, 0)^\top$

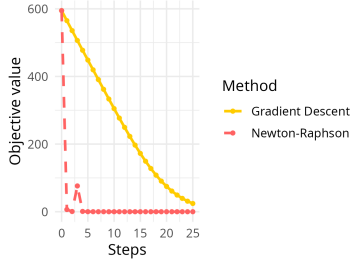
After 5 Newton steps – the last three are no longer visible, the iterates already sit on $\boldsymbol{x}^*$:

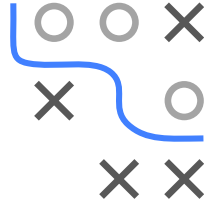




NEWTON-RAPHSON VS. GD ON ROSENBROCK FUNCTION

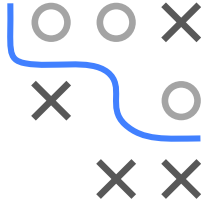






Newton-Raphson has much better convergence speed here

NEWTON-RAPHSON: STEP SIZE

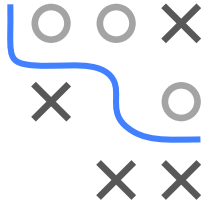


- **Relaxed/Damped Newton-Raphson:** Use step size $\alpha > 0$ with

$$\boldsymbol{x}^{[t+1]} = \boldsymbol{x}^{[t]} + \alpha \mathbf{d}^{[t]}$$

to satisfy Wolfe conditions (or just Armijo rule)

DISCUSSION



Advantage:

- For f sufficiently smooth:

Newton-Raphson converges *locally* quadratically (i.e., for starting points close enough to stationary point)

Disadvantage:

- For “bad” starting points:

Newton-Raphson may diverge